采用 Pthread 对 BMP 图片进行卷积

1. 实验目的

本次实验的目的是通过计算密集型的 BMP 图像处理任务，掌握和实践共享内存架构下的并行编程模型，包括以下核心内容：

a) **掌握 Pthread 并行编程：**深入理解共享内存架构下的核心概念与编程范式,利用 POSIX Threads(Pthread)库作为实现线程级并行与协作的手段，掌握线程的创建、参数传递、同步与资源回收机制

b) **实现复杂的并行算法：**针对 BMP 图像高斯卷积(使用 σ=1.5 的 5×5 高斯核)这一大规模像素计算任务，不依赖第三方图像库，从底层设计并实现一个高效、正确的多线程并行卷积算法

c) **掌握关键并行策略与优化技术：**运用按行区域分解策略，将图像数据划分为互不重叠的行块，以解决多线程间的负载均衡问题，运用补零技术正确处理图像边缘像素，并进一步探索可分离卷积与栅栏同步技术，通过降低算法复杂度与构建两阶段并行流水线，大幅提升计算性能

d) **验证并行计算的正确性：**确保并行计算输出的 output.bmp 图像文件及 matrix.txt 验证文档，能通过作业提供的验证程序与标准 OpenCV 算法结果进行比对，保证 MSE 极低且相似度达到 99%以上

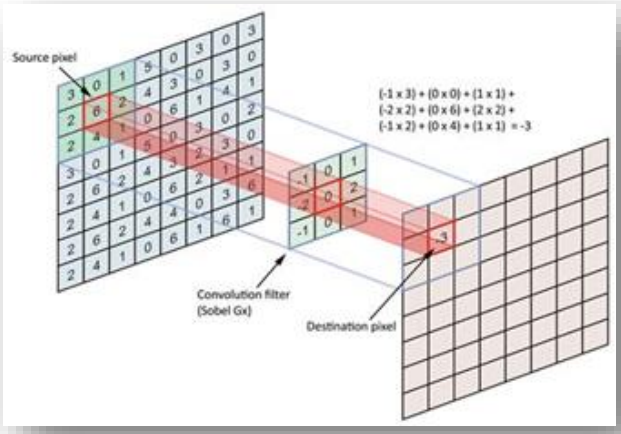
e) **分析并行性能指标：**通过测量程序在 1 线程(串行)、2 线程、4 线程配置下的总运行时间，来量化评估并行化带来的性能提升；计算并对比不同线程数下的并行加速比与并行效率,并基于实验数据分析内存带宽竞争、线程调度开销等制约性能的关键瓶颈

2. 并行策略与算法设计（基础版）

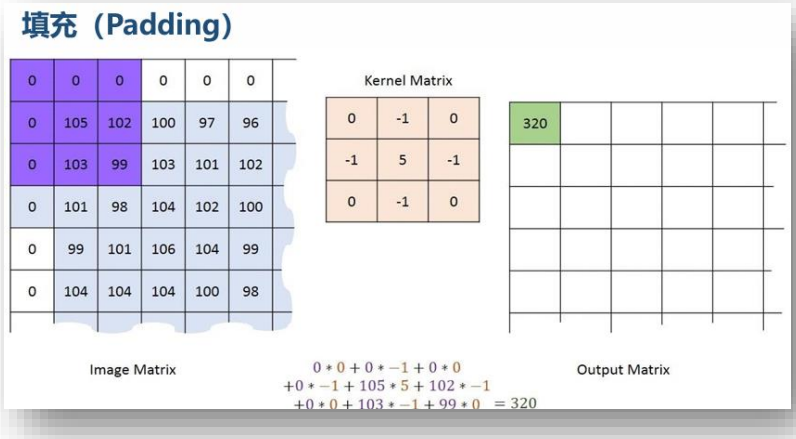
a) 算法原理与设计

本次实验的核心计算任务是基于空间域的图像滤波，旨在通过底层算法实现对 BMP 图像的精确处理

- **高斯卷积核**：实验统一采用尺寸为 5·5 的高斯卷积核，标准差设定为 1.5。该权重矩阵呈现中心高、四周低的分布特性，用于对图像进行平滑处理以消除噪声
- **卷积运算逻辑**：对于图像中的每一个像素点，算法选取以其为中心的 5x5 邻域窗口，将窗口内的 25 个像素 R、G、B 分量值分别与对应的核权重进行乘加运算



- **边界处理策略**：由于卷积窗口在处理图像边缘时会超出实际物理边界，本实验采用零填充策略。即当访问坐标超出图像范围时，默认该位置的像素值为 0，保证了卷积操作在全图范围内的统一性，且维持输出图像尺寸不变

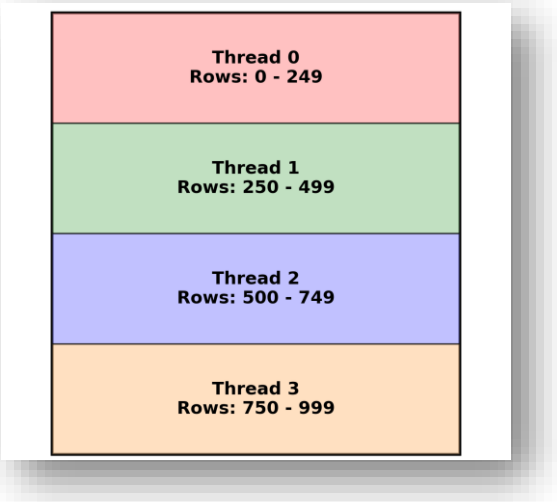


- **数值规约与截断**：卷积计算产生的中间结果为浮点数，为了符合 BMP 格式标准，需将结果进行四舍五入，并强制截断至 0 到 255 的整数闭区间内，防止颜色溢出

b）并行化策略

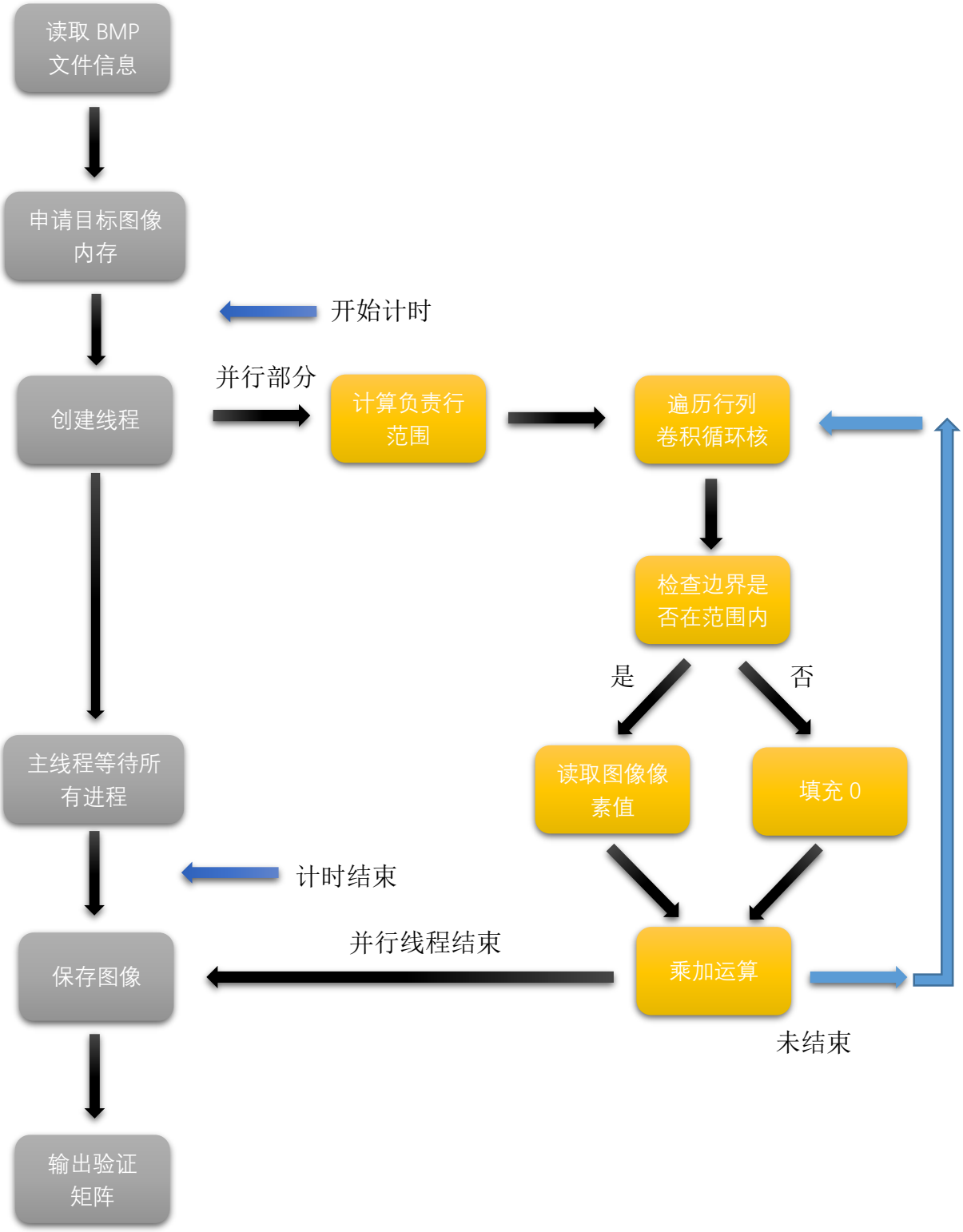
为了充分利用多核 CPU 的计算资源，本实验基于共享内存架构设计了高效的数据并行方案

- **按行区域分解：**实验采用静态任务划分策略，将二维图像数据在垂直方向上进行切分。设图像高度为 H，启动的线程总数为 N，系统将图像均匀划分为 N 个连续的行块，每个线程负责计算其中约 H/N 行的数据。例如，线程 id 为 k 的线程负责处理第 k 个行块



- **无锁并行机制：**算法设计确保了极高的数据独立性。在计算过程中，所有线程共享读取同一份源图像内存(只读)，并将计算结果写入目标图像内存中互不重叠的区域(只写)。由于不存在多个线程同时修改同一内存地址的竞态条件，系统无需引入互斥锁或信号量等同步机制，消除了上下文切换开销，最大化了并行加速比
- **内存布局与访问优化：**程序不依赖第三方库，直接解析 BMP 二进制文件结构，将二维像素矩阵线性映射为内存中的一维连续数组。通过计算偏移量直接访问像素，相比于多级指针结构，这种连续存储显著提高了空间局部性，使得 CPU 能够更高效地利用缓存预取数据，大幅降低了访存延迟

c）流程图


读取 BMP 文件信息
申请目标图像内存
开始计时
并行部分
创建线程
计算负责行范围
遍历行列卷积循环核
检查边界是否在范围内
是
否
读取图像像素值
填充 0
主线程等待所有进程
计时结束
并行线程结束
保存图像
乘加运算
未结束
输出验证矩阵

3. 核心代码分析

a) 自定义数据结构与参数传递

为了实现对 BMP 文件的底层读写及线程参数的高效传递，定义了以下关键结构体：

- **BMP 文件头结构体：**通过 BMPFileHeader 与 BMPInfoHeader 结构体，严格按照 BMP 文件格式标准定义了 1 字节对齐的内存布局，程序能够通过文件流直接读取图像的宽度、高度及位深信息
- **线程参数结构体：**定义 ThreadData 结构体用于主线程与工作线程间的通信，该结构体封装了线程 ID、线程总数、源图像指针以及目标图像指针，这种设计避免了全局变量的使用，增强了代码的封装性与线程安全性

```
struct ThreadData {
    int thread_id;
    int num_threads;
    BMPImage* srcImg;
    BMPImage* destImg;
    double cost_time; // 存储该线程的运行时间
};
```

b) 并行计算核心逻辑

convolution_routine 函数是并行执行的入口，其核心逻辑实现了基于行分解的任务划分与卷积运算

- **任务划分：**函数首先根据传入的 thread_id 与 num_threads 计算当前线程负责处理的行范围。通过简单的整数除法将图像高度H映射为连续的行块，实现了静态负载均衡

```
    // 按行划分任务
    int chunk_size = height / data->num_threads;
    int start_row = data->thread_id * chunk_size;
    int end_row = (data->thread_id == data->num_threads - 1) ? height :
start_row + chunk_size;
```

- **卷积运算与边界处理：**在遍历像素的循环内部，程序实现了标准的二维离散卷积，对于 5x5 邻域内的每一个点，程序显式进行了边界检查以实现 Zero Padding 策略。

界内处理：若邻域坐标(nx，ny)位于图像范围内，则读取对应像素值并与高斯核权重相乘

越界处理：若坐标超出图像边界，则跳过读取操作（隐式地视为 0），从而保证了边缘计算的合法性

```
// 卷积操作
    for (int y = start_row; y < end_row; ++y) {
        for (int x = 0; x < width; ++x) {
            float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

            // 应用 5x5 核
            for (int ky = -K_RADIUS; ky <= K_RADIUS; ++ky) {
                for (int kx = -K_RADIUS; kx <= K_RADIUS; ++kx) {
                    int ny = y + ky;
                    int nx = x + kx;

                    // 边界处理：如果越界，则视为 0 (Zero Padding)
                    // 作业要求：边缘节点填 0
                    if (ny >= 0 && ny < height && nx >= 0 && nx < width)
{
                        Pixel& p = data->srcImg->data[ny * width + nx];
                        float w = KERNEL[ky + K_RADIUS][kx + K_RADIUS];
                        sum_r += p.r * w;
                        sum_g += p.g * w;
                        sum_b += p.b * w;
                    }
                    // else: implicit +0
                }
            }

            // 写回结果
            int idx = y * width + x;
            data->destImg->data[idx].r =
(unsignedchar)std::min(std::max(sum_r, 0.0f), 255.0f);
            data->destImg->data[idx].g =
(unsignedchar)std::min(std::max(sum_g, 0.0f), 255.0f);
            data->destImg->data[idx].b =
(unsignedchar)std::min(std::max(sum_b, 0.0f), 255.0f);
        }
    }
```

4. 实验结果分析

a）正确性检验

```
wyj2350489@ps:~/PThread/测试程序$ ./pthread 1
Starting convolution with 1 threads...
Thread 0 calculation time: 1.060686 seconds
Total Parallel Execution Time: 1.06092 seconds
Image saved to output.bmp
Kernel and matrix data saved to matrix.txt
wyj2350489@ps:~/PThread/测试程序$ g++ test.cpp -o test_checker `pkg-config --cflags --libs opencv4`
wyj2350489@ps:~/PThread/测试程序$ ./test_checker timg.bmp output.bmp
[PASS] Shape test passed!
[PASS] Similarity test passed!
MSE result: 0.55
Similarity rate: 99.71%
```

1 核

```
wyj2350489@ps:~/PThread/测试程序$ ./pthread 2
Starting convolution with 2 threads...
Thread 1 calculation time: 0.521185 seconds
Thread 0 calculation time: 0.564060 seconds
Total Parallel Execution Time: 0.564305 seconds
Image saved to output.bmp
Kernel and matrix data saved to matrix.txt
wyj2350489@ps:~/PThread/测试程序$ ./test_checker timg.bmp output.bmp
[PASS] Shape test passed!
[PASS] Similarity test passed!
MSE result: 0.55
Similarity rate: 99.71%
```

2 核

```
wyj2350489@ps:~/PThread/测试程序$ ./pthread 4
Starting convolution with 4 threads...
Thread 1 calculation time: 0.264098 seconds
Thread 3 calculation time: 0.273832 seconds
Thread 0 calculation time: 0.300053 seconds
Thread 2 calculation time: 0.334926 seconds
Total Parallel Execution Time: 0.33535 seconds
Image saved to output.bmp
Kernel and matrix data saved to matrix.txt
wyj2350489@ps:~/PThread/测试程序$ ./test_checker timg.bmp output.bmp
[PASS] Shape test passed!
[PASS] Similarity test passed!
MSE result: 0.55
Similarity rate: 99.71%
```

4 核

b）性能测试数据

线程数（p）	各线程独立耗时（s）	总运行时间（s）	加速比（Sp）	并行效率（Ep）
1（串行）	T0：1.061	1.061	1.00	100%
2	T0：0.564 T1：0.521	0.564	1.88	94.0%
4	T0：0.300 T1：0.264 T2：0.335 T3：0.274	0.335	3.17	79.2%

c）结果分析

1．加速比分析：

从 1 线程增加到 4 线程，程序的运行时间从 1.061 秒大幅下降至 0.335 秒，加速比达到 3.17，这表明基于行分解的数据并行策略有效地利用了多核 CPU 资源，显著提升了图像处理的吞吐量

2．负载均衡与短板效应：

观察“各线程独立耗时”数据可以发现，尽管任务按行数均匀分配，但不同线程的实际执行时间存在显著差异，在 4 线程测试中，最快线程(T1)仅耗时 0.264s，而最慢线程(T2)耗时 0.335s，两者相差约 26%

并行程序的总耗时(0.335s)严格取决于最慢的那个线程。这种差异可能源于操作系统调度波动、不同内存区域的访存延迟差异以及 CPU 缓存命中的不均匀性

3．效率衰减分析：

随着线程数增加，并行效率从 94.0%下降至 79.2%。除了上述负载微小不均导致的等待时间外，内存带宽瓶颈是主要原因。图像卷积属于高频访存任务，当 4 个核心同时争抢内存总线时，导致了访存延迟增加，限制了线性加速比的达成

5. 优化策略与算法改进

尽管基础版程序通过数据并行获得了显著的加速比，但在算法层面，传统的二维直接卷积仍存在计算冗余。为了进一步挖掘并行系统的性能潜力，本实验引入了基于数学特性的算法级优化方案，旨在降低计算复杂度并提升访存效率

a） 优化策略：可分离卷积

高斯卷积核本质上是由两个一维高斯向量（水平方向与垂直方向）的外积生成的，具有线性可分性。基于此特性，本实验将原本的一步二维卷

积重构为两步一维卷积

- 第一步（水平滤波）：使用 1×5 的行向量对图像进行水平方向卷积
- 第二步（垂直滤波）：使用 5×1 的列向量对上一步的中间结果进行垂直方向卷积

b) **并行流水线实现**

可分离卷积引入了数据依赖性，即垂直卷积必须等待水平卷积完成后方可进行。为了在多线程环境下正确执行，实验使用了 Pthread Barrier 技术构建了两阶段并行流水线

阶段一：所有线程并行计算分配到的行块的水平卷积，并将结果存入高精度的浮点型中间缓冲区 tempBuffer

栅栏同步：调用 pthread_barrier_wait。所有线程在此处阻塞，直到所有 N 个线程都完成阶段一的工作，确保了中间数据的完整性

阶段二：同步解除后，所有线程再次并行读取中间缓冲区，执行垂直卷积并输出最终结果

线程数 (p)	各线程独立耗时 (s)	总运行时间 (s)	加速比 (Sp)	并行效率 (Ep)
1（串行）	T0: 0.3975	0.4064	1.00	100%
2	T0: 0.1979 T1: 0.2020	0.2022	2.00	100%
4	T0: 0.1077 T1: 0.1091 T2: 0.1095 T3: 0.1239	0.1242	3.27	81.8%

同核对比提升：在 4 线程下，优化版(0.124s)比基础版(0.335s)快了约 2.7 倍，这与计算量减少 60% 的理论预测高度一致

单核即胜多核：优化版的单线程性能(0.406s)已经接近基础版的 4 线程性能(0.335s)，这证明了算法优化（降低复杂度）往往比单纯的硬件堆叠（增加核心）更有效

精度提升：优化版的 MSE 为 0.57（与基础版 0.55 持平或微优，取决于浮点截断误差），证明了流水线逻辑的正确性

6. 总结

本次实验成功实现了基于 Pthread 的 BMP 图像并行卷积

a) **基础实现**：验证了数据并行策略的有效性，在 4 核环境下取得了 3.17 倍的加速比

b) **进阶优化**：创新性地实现了可分离卷积算法，结合 pthread_barrier 解决了并行流水线的同步问题，将运行速度进一步提升了 2.7 倍

c) **心得体会**：并行计算不仅仅是把任务分给更多人做，更重要的是如何少做无用功，算法优化以及如何协调工作，同步与负载均衡，在高访存应用中，算法复杂度的降低比单纯增加线程数带来的收益更为显著。